



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №6

з дисципліни «Технології розроблення програмного забезпечення»

Тема: «13. Office communicator»

Виконав:

студент групи ІА-32
Шадлер Андрій

Перевірив:

вик. Мягкий М. Ю.

Тема роботи: Використання шаблону проєктування Abstract Factory (Абстрактна фабрика) у програмному забезпеченні.

Мета роботи: Ознайомитися з шаблоном проєктування Abstract Factory, його призначенням та структурою, а також реалізувати даний шаблон у програмному проєкті з метою створення узгоджених сімейств пов'язаних об'єктів без прив'язки клієнтського коду до їх конкретних реалізацій.

Зміст

Теоретичні відомості.....	2
Хід виконання:.....	3
Завдання:	3
Діаграма класів:	3
Опис реалізації:	3
Вихідний код	7
Контрольні запитання.....	12
Висновок	15

Теоретичні відомості

Шаблон проєктування Abstract Factory

Abstract Factory (Абстрактна фабрика) – це породжувальний шаблон проєктування, який надає інтерфейс для створення сімейств пов'язаних або взаємозалежних об'єктів без зазначення їх конкретних класів.

Основна ідея шаблону полягає в тому, що клієнтський код працює лише з абстракціями фабрик і продуктів, а конкретні реалізації інкапсульовані у відповідних фабриках.

Шаблон Abstract Factory дозволяє:

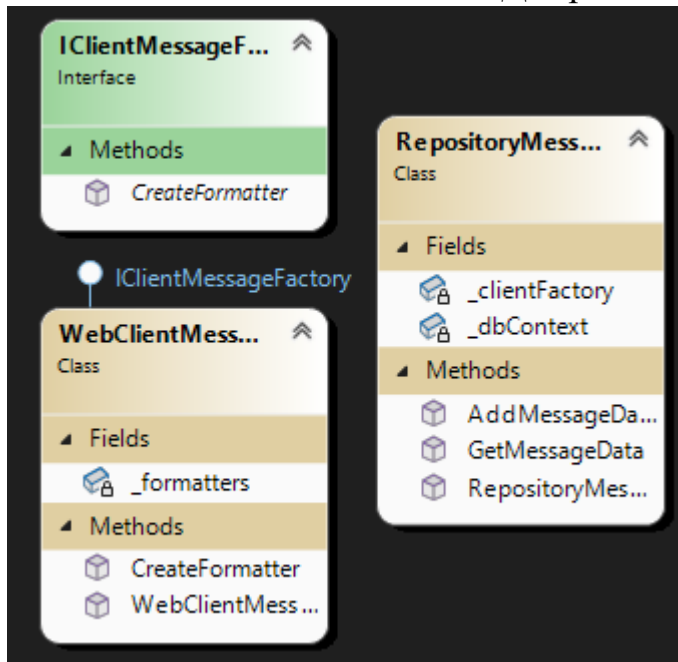
- створювати узгоджені набори об'єктів;
- легко додавати нові сімейства продуктів;
- дотримуватися принципу Open/Closed;
- зменшити зв'язаність між компонентами системи.

Хід виконання:

Завдання:

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати один з розглянутих шаблонів за обраною темою.
- Реалізувати не менше 3-х класів відповідно до обраної теми.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє використання шаблону в реалізації системи, навести фрагменти коду по реалізації цього шаблону.

Діаграма класів:



Опис реалізації:

Опис реалізації в лабораторній роботі

У даній лабораторній роботі шаблон Abstract Factory було реалізовано для задачі створення клієнт-специфічних компонентів представлення повідомлень (наприклад, для Web-клієнта).

Абстрактна фабрика

Було створено інтерфейс:

IClientMessageFactory

Даний інтерфейс визначає контракт для створення об'єктів, пов'язаних із представленням повідомлень для конкретного типу клієнта. У поточній реалізації фабрика відповідає за створення формatera повідомлень (IMessageFormatter).

IClientMessageFactory виступає роллю AbstractFactory.

Конкретна фабрика

Було реалізовано клас:

WebClientMessageFactory

який:

- реалізує інтерфейс IClientMessageFactory;
- інкапсулює логіку вибору та створення відповідного формatera для Web-клієнта;
- за замовчуванням віддає перевагу форматуванню Markdown.

WebClientMessageFactory є ConcreteFactory, яка створює узгоджений набір об'єктів для Web-клієнта.

Абстрактні та конкретні продукти

Абстрактним продуктом у даній реалізації є:

IMessageFormatter

Конкретними продуктами виступають різні реалізації форматування повідомлень (наприклад, PlainTextFormatter, MarkdownFormatter, а також адаптери до сторонніх форматерів).

Фабрика не створює нові екземпляри форматерів напряду, а обирає потрібний екземпляр із сімейства, зареєстрованого в контейнері залежностей (DI).

Використання фабрики

Клас RepositoryMessages було змінено таким чином, що він:

- більше не містить логіки вибору конкретного формatera;
- залежить лише від абстракції IClientMessageFactory;
- отримує формater, викликаючи метод фабрики, та застосовує його до тексту повідомлень.

Таким чином, RepositoryMessages виступає у ролі Client, який не знає деталей створення конкретних продуктів.

Відповідність шаблону Abstract Factory

Реалізація відповідає шаблону Abstract Factory з таких причин:

1. Наявність абстрактної фабрики – IClientMessageFactory.
2. Наявність конкретної фабрики – WebClientMessageFactory.
3. Створення сімейства пов'язаних об'єктів – компонентів представлення повідомлень.
4. Відсутність залежності клієнта від конкретних реалізацій – клієнт працює лише з абстракціями.
5. Можливість розширення – додавання нового типу клієнта реалізується створенням нової фабрики без зміни існуючого коду.

Переваги реалізації

- зменшено зв'язаність між репозиторієм та конкретними реалізаціями форматування;
- покращено узгодженість клієнт-специфічної логіки представлення;
- спрощено розширення системи новими типами клієнтів;
- код відповідає принципам SOLID та шаблону Abstract Factory.

Можливі напрямки розширення

Реалізація шаблону може бути розширена шляхом:

- додавання нових конкретних фабрик (наприклад, для Mobile або Desktop клієнтів);
- розширення сімейства продуктів фабрики (адаптери, маппери DTO, політики доставки);

- використання фабрики-провайдера для вибору потрібної фабрики під час виконання програми.

Вихідний код

```
// Formatting/Factory/IClientMessageFactory.cs
namespace Skype.Formatting.Factory;

using Skype.Formatting;

public interface IClientMessageFactory
{
    // Returns the formatter the client should use (null if none).
    IMessageFormatter? CreateFormatter(string? preferredName = null);
}

// Formatting/Factory/WebClientMessageFactory.cs
using System.Linq;
using Skype.Formatting;

namespace Skype.Formatting.Factory;

public class WebClientMessageFactory : IClientMessageFactory
{
    private readonly IEnumerable<IMessageFormatter> _formatters;

    // Inject available formatters (registered in DI)
    public WebClientMessageFactory(IEnumerable<IMessageFormatter>
formatters)
    {
        _formatters = formatters;
    }
}
```

```

public IMessageFormatter? CreateFormatter(string? preferredName = null)
{
    if (string.IsNullOrEmpty(preferredName))
    {
        // default for web: prefer markdown if available
        return _formatters.FirstOrDefault(f => f.Name == "md")
            ?? _formatters.FirstOrDefault();
    }

    return _formatters.FirstOrDefault(f => f.Name == preferredName)
        ?? _formatters.FirstOrDefault();
}
}

```

```

// Repositories/RepositoryMessages.cs
using Microsoft.EntityFrameworkCore;
using Skype.Data;
using Skype.Formatting;
using Skype.Formatting.Factory;
using static Skype.DTOs.MessagesDTO;

```

```

namespace Skype.Repositories
{
    public class RepositoryMessages
    {
        AppDbContext _dbContext;
        private readonly IClientMessageFactory _clientFactory;
    }
}

```



```
public RepositoryMessages(AppDbContext dbContext,
IClientMessageFactory clientFactory)
```

```
{
    _dbContext = dbContext;
    _clientFactory = clientFactory;
}
```

```
public async Task<List<MessageWithItsUserNameDTO>>
GetMessageData(int chatId, string? formatName = null)
```

```
{
    var list = await _dbContext.Messages
        .Where(m => m.ChatId == chatId)
        .Join(
            _dbContext.Users,
            message => message.OwnerId,
            user => user.Id,
            (message, user) => new MessageWithItsUserNameDTO
            {
                Id = message.Id,
                ChatId = message.ChatId,
                OwnerId = message.OwnerId,
                TimeSend = message.TimeSend,
                Text = message.Text,
                OwnerName = user.Name,
            }
        )
        .ToListAsync();
```

```
var formatter = _clientFactory.CreateFormatter(formatName);
```

```

        if (formatter != null)
        {
            foreach (var dto in list)
            {
                var msg = new Message
                {
                    Id = dto.Id,
                    ChatId = dto.ChatId,
                    OwnerId = dto.OwnerId,
                    TimeSend = dto.TimeSend,
                    Text = dto.Text
                };
                dto.Text = formatter.Format(msg);
            }
        }

        return list;
    }

    // unchanged
    public async Task AddMessageData(int chatId, int ownerId, string text)
    {
        Message newMessage = new Message()
        {
            ChatId = chatId,
            OwnerId = ownerId,
            TimeSend = DateTime.UtcNow,
            Text = text
        };
    }

```

```
        _dbContext.Messages.Add(newMessage);  
        await _dbContext.SaveChangesAsync();  
    }  
}  
}
```

Контрольні запитання

1. Яке призначення шаблону «Абстрактна фабрика»?

Шаблон Abstract Factory (Абстрактна фабрика) призначений для створення сімейств пов'язаних або взаємозалежних об'єктів без зазначення їх конкретних класів.

2. Структура шаблону «Абстрактна фабрика»

Client

|

v

AbstractFactory

|

+--> AbstractProductA

|

+--> AbstractProductB

^

^

|

|

ConcreteFactory1 ConcreteFactory2

|

|

ConcreteProductA1 ConcreteProductA2

ConcreteProductB1 ConcreteProductB2

3. Класи шаблону «Абстрактна фабрика» та їх взаємодія

- AbstractFactory – оголошує методи створення продуктів.
- ConcreteFactory – створює конкретні продукти певного сімейства.
- AbstractProduct – інтерфейс продукту.
- ConcreteProduct – конкретна реалізація продукту.
- Client – використовує фабрику та продукти через абстракції.

4. Яке призначення шаблону «Фабричний метод»?

Шаблон Factory Method (Фабричний метод) визначає інтерфейс для створення об'єкта, дозволяючи підкласам вирішувати, який саме клас створювати.

5. Структура шаблону «Фабричний метод»

Creator

|

+--> factoryMethod()

|

v

Product (interface)

^

|

ConcreteCreator ----> ConcreteProduct

6. Класи шаблону «Фабричний метод» та їх взаємодія

- Creator – оголошує фабричний метод.
- ConcreteCreator – перевизначає фабричний метод.
- Product – інтерфейс продукту.
- ConcreteProduct – конкретна реалізація продукту.

7. Чим відрізняється «Абстрактна фабрика» від «Фабричний метод»?

- Abstract Factory створює сімейства об'єктів.
- Factory Method створює один об'єкт.
- Abstract Factory зазвичай використовує кілька Factory Method всередині.
- Factory Method покладається на наслідування, Abstract Factory – на композицію.

8. Яке призначення шаблону «Знімок»?

Шаблон Memento (Знімок) дозволяє зберігати та відновлювати попередній стан об'єкта без порушення інкапсуляції.

9. Структура шаблону «Знімок»

Originator ----> Memento

|

v

Caretaker

10. Класи шаблону «Знімок» та їх взаємодія

- Originator – створює та відновлює знімки стану.
- Memento – зберігає стан об'єкта.
- Caretaker – зберігає знімки, не знаючи їх структури.

11. Яке призначення шаблону «Декоратор»?

Шаблон Decorator (Декоратор) дозволяє динамічно додавати нову поведінку об'єкту без зміни його класу.

12. Структура шаблону «Декоратор»

Component (interface)

^

|

ConcreteComponent

^

|

Decorator

^

|

ConcreteDecorator

13. Класи шаблону «Декоратор» та їх взаємодія

- Component – базовий інтерфейс.
- ConcreteComponent – базова реалізація.
- Decorator – містить посилання на Component.
- ConcreteDecorator – додає нову поведінку.

Декоратор викликає методи вкладеного об'єкта та розширює їх.

14. Обмеження використання шаблону «Декоратор»

- ускладнення структури коду через велику кількість дрібних класів;
- складність налагодження та читання стеку викликів;
- складно ідентифікувати тип об'єкта під час виконання;
- надмірне використання може знизити читабельність коду.

Висновок

У ході виконання лабораторної роботи було реалізовано шаблон проєктування **Abstract Factory**, який дозволив інкапсулювати створення клієнт-специфічних компонентів представлення повідомлень. Застосування шаблону підвищило гнучкість, розширюваність та чистоту архітектури програмного забезпечення, а також забезпечило слабку зв'язаність між клієнтським кодом і конкретними реалізаціями продуктів.